PSAI_091

Lollipop: sBPF Runtime as a Service on PolkaVM

- **Team Name:** Lollipop Builders
- **Payment Details:**
  - **DOT**: 13W1E9hU8oKgVL1uaAGYTFLY9G1q2YJTjrVN5YEhUoXM4QHT
  - **Payment**: 13W1E9hU8oKgVL1uaAGYTFLY9G1q2YJTjrVN5YEhUoXM4QHT (USDC), 50% USDC and 50% in vested DOT
- **Level:** 3

Project Overview :page_facing_up:

Overview

Tag line: Lollipop - modular framework for Solana and SVM, based on Polkadot JAM and PolkaVM.

Brief description: Lollipop is a provider of modular solutions for Network Extensions/Solana L2/Rollup with diverse execution environments that can broadly be categorized as middlewares and instances. As opposed to Layer 2 solutions, Network Extensions does not result in liquidity or user fragmentation. Lollipop is an extension of Solana's monolithic infrastructure. NES environments can include TEEs, co-processors, zk-encryption, FHE, trusted solver networks, AI agents and more, even in non-blockchain contexts. Lollipop Instances and Middlewares do not rely on cross-chain technologies for integration with Solana. Instead, they interconnect via Solana itself, utilizing program delegation, preserving direct connectivity without bridges.

Integration into Polkadot: Polkadot JAM in emulating SVM execution with PolkaVM, which is RISK-V level virtual-machine, so users may use SVM programs with Polkadot's JAM, just like Mac users can run windows app with emulators like Parallels Desktop. With this being successful, the Execution Environment (VM) Fragmentation will be gone completely. sBPF Runtime as a Service in PolkaVM - is a middleware that will enable running SVM-related environments, taking full advantage of PVM and increasing Polkadot utilisation through synergies with the Solana and SVM-based ecosystems.

Motivation: The Lollipop team is made up of members who are deeply immersed in Polkadot, Solana, and other ecosystems. Being inspired by the opportunities presented by the Polkadot JAM along with understanding the functional bottlenecks for Solana led us to the vision for Lollipop. We came to the conclusion that a modular framework based on Polkadot technologies with the widest customization possibilities is ideal for both the Solana ecosystem and for running individual spaces (runtimes) based on PVM combined with sBPF.

Project Details

Overview of the Technology Stack - **PolkaVM** – The execution environment for running sBPF programs. - **sBPF Runtime** – Embedded sBPF interpreter to support Solana smart contracts. - **Rust** – Primary development language for modifying the sBPF interpreter and integrating it with PolkaVM. - **jam-pvm-common** – Provides common utilities for interaction with PolkaVM. - **Solana Program Library (SPL) Support** – To enable compatibility with existing Solana contracts.

PoC/MVP or Other Relevant Prior Work - **Concept Validation:** Discussions with Dr. Gavin Wood on feasibility. - **Technical Research:** Published article "sBPF Program Execution on PVM - HackMD". - **On-going Work:** Adaptation of 'anza-xyz/sbpf' for no_std execution. - **Initial Feasibility Testing:** Binary profiling and testing of execution overhead.

Data Models / API Specifications of the Core Functionality

The sBPF Runtime Feasibility & Prototype Development in PolkaVM aims to evaluate whether sBPF programs can be executed inside PolkaVM with modified sBPF runtime, with a minimal prototype that includes basic contract execution and result storage, the API specifications focus on: 1. Submitting an sBPF program for execution in PolkaVM 2. Retrieving and storing execution results 3. Validating the feasibility of state management

Execution API (Minimal Prototype Scope) - Purpose: Allows an off-chain service to submit a precompiled sBPF program for execution inside PolkaVM. - Input: - 'program_id': Unique identifier of the sBPF contract - 'execution_params': Parameters for the execution (e.g., function input, gas limit) - Output: - Execution logs (`stdout`, `errors`, `syscall output`) - State difference (`{storage_addr: old → new}`)

State Diff & Result Storage API - Purpose: Stores execution results outside of PolkaVM for later verification. - Input: - `execution_id`: Unique execution session ID - `state_diff`: Key-value pairs of changed state values - `logs`: Execution logs for debugging - Output: - Confirmation of successful storage

> **Please Note:** - The state difference is stored in an external DB rather than committed to blockchain storage. - This ensures minimal modifications to PolkaVM while still capturing execution feasibility.

Documentation of Core Components, Protocols, Architecture: The architecture for this feasibility study and prototype development is designed to: - Execute a basic sBPF contract (counter program) inside PolkaVM - Store the execution results externally - Validate whether execution is feasible within PolkaVM's constraints (4MB limit, memory access, syscalls, etc.)

We have justified the need for a modular solution for Solana in detail in a number of our researches and observations: - https://hackmd.io/@clearloop/rk6JhtdK1g - https://www.lollipop.builders/Lollipop%20Network%20Extensions%20Report.pdf - https://www.lollipop.builders/research - https://medium.com/lollipop-builders/trump-and-melania-transactions-failure-investigation-b2a5c54d2021 - https://medium.com/lollipop-builders/how-lollipop-is-turning-solana-into-a-modular-playground-for-developers-c715262a5a1e

Business-oriented activities (marketing, business planning), events or outreach: At the moment, the test environment is in the process of being deployed. We are currently working with Mantis, RWA business, perp dexes and we are also developing a presence in the gaming sector. Lollipop also has arrangements with RWA business, RWA business which will issue tokens and be regulated by the HK authorities based on Solar power plant receivables for the next ten years. We are also in the process of discussing engagement with

marketing agencies such as Market Across and Cointelegraph. The other most important domain for Lollipop is PayFi. Demand for crypto-based infrastructure and stablecoins is growing from FinTech and banking. This includes the creation of new financial primitives for payment functions with financial services based on blockchain technology and smart contracts. For PayFi, it is critically important that transaction costs remain minimal and predictable. To fully deploy PayFi infrastructure, it will be necessary to implement Lollipop modules, which will facilitate off-chain processing with predictable costs combined with all the benefits of JAM and PolkaVM. Additionally, NE can assist in ensuring compliance and regulatory adherence for payment applications across different regions by utilising the Solana Permissioned Environment with JAM.

Ecosystem Fit

Where and how does your project fit into the ecosystem? The sBPF Runtime as a Service in PolkaVM will bring value to Polkadot by connecting to the huge ecosystem and user base of SVM-based spaces (including Solana and other ecosystems like Eclipse, Termina and so on). PVM capabilities will enhance SVM frameworks by introducing additional functionality. This implies many different network effects and utilisation by shifting both user and economic load to the Polkadot tech stack, with the introduction of fundamentally new economic models targeting enterprise customers, institutional players, users, and infrastructure providers.

Who is your target audience (parachain/dapp/wallet/UI developers, designers, your own user base, some dapp's userbase, yourself)? In the context of "sBPF Runtime as a Service in PolkaVM", the proposed solution targets the user and developer base of Solana, Polkadot, and SVM-based L1/L2 ecosystems. The Runtimes/Environments are designed for different use cases: - **Solana-related dApps and Protocols:** Off-chain process handling will enable the creation of highly customizable execution environments tailored to various requirements and tasks, with direct connectivity to Solana. - **Isolated customizable execution environments:** Designed for diverse applications such as Perp DEXes, Gaming, Oracles, enterprise, and government solutions. - **Developers:** Our solution expands development opportunities by seamlessly bridging different developer cohorts through the introduction of new tools and primitives. - **Users:** the customizability of runtimes will open up new capabilities not available to institutional and enterprise customers in Solana and SVM forks - **Polkadot ecosystem participants:** validators, delegators, developers.

Are there any other projects similar to yours in the Substrate / Polkadot / Kusama ecosystem? Previously, the Polkadot community considered proposals for deploying SVM on Parachains **(link)**. However, this approach involved significant trade-offs due to the architecture of Polkadot 1.0 and required a compatibility layer for the Substrate node to function as a Solana node. The constraints related to performance limitations and the need to develop an ecosystem around a Solana fork within a Parachain—without providing tangible benefits to either the Solana or Polkadot ecosystems—rendered this idea impractical. In the case of Lollipop, JAM is used as the primary execution layer. Moreover, it is an application-specific implementation rather than merely a fork of Solana, as in the approach described above.

Are there any projects similar to yours in related ecosystems? The approach presented by Lollipop using PolkaVM, as far as we know at the moment, has no alternatives also in other ecosystems and is unique. Restaking platforms can be considered somewhat conceptually related, as they enable the deployment of Actively Validated Services (AVS) within specific environments. Some Solana/SVM-related projects refer to themselves as Network Extensions, but in most cases, they are standard rollups (e.g., Soon SVM as an optimistic rollup, Termina as RaaS) or sidechains (e.g., Neon EVM) without seamless integration into the Solana.

Team :busts_in_silhouette:

Team members

- Name of team leader: Qinwen Wang
- Names of team members: Kyle Gu, Tianyi , Stepan Soin, Wu, James Wade

Contact

- **Contact Name:** Qinwen Wang
- **Contact Email:** qinwen@lollipop.builders
- **Website:** https://www.lollipop.builders/

Legal Structure

- **Registered Address:** Chambers, Road Town, Tortola VG1110, British Virgin Islands
- **Registered Legal Entity:** Lollipop Builder Ltd.

Team's experience

The Lollipop team consists of experienced cryptographers, developers and crypto builders and is focused on building Network Extensions for Solana using Polkadot technologies. - **Qinwen Wang:** CEO, Founder. 4 years in Web3 foundation; Producer of "Virtual Niche" - the World's first Crypto Art NFT Museum Exhibition; Curator and lecturer for Christie's NFT 101 HK & London; Curator and collector of Sotheby's metaverse auction; Angel investor, Forbes 30U30. - **Kyle Gu:** Founder's office. MSC in Actuarial Science & PD in Computer Science of University of Kent; Over 8 Years of development operation experiences for crypto projects; Prev Magnet Network Project Manager; Prev Darwinia Network BD Manager. - **Tianyi:** Head of JAM Implementation. Author of Spacejam Network, Zink language contributor, prev developer and top 5 contributors of GEAR, prev developer in Chainsafe. - **Stepan Soin:** Head of Research. 4 years General manager and ecosystem researcher at Avalanche, Researcher and Coordinator at Dehales Capital, VC Investment analytics Cryptomeria Capital. - **Wu:** Lead developer. Expert in VM compilation and cross-platform compatibility, specializing in EVM, WASM, and BPF. Designs efficient toolchains for seamless smart contract execution across Ethereum, Polkadot, and Solana. Proficient in Rust, developing blockchain nodes, VM runtimes, and cross-chain protocols with a focus on security, efficiency, and high-performance computing. - **James Wade:** Developer. Expert in blockchain architecture and protocols across Bitcoin, Ethereum, Polkadot, and Solana. Specializes in cryptographic algorithms and cross-chain interoperability, designing secure and efficient multi-chain asset exchange and data transfer solutions for decentralized environments.

Team Code Repos

- https://github.com/HONGYI-SD
- https://github.com/clearloop

- https://github.com/lei-1993

Team LinkedIn Profiles (if available)

- https://www.linkedin.com/in/qinwenwang/
- https://www.linkedin.com/in/kyle-gu/
- https://www.linkedin.com/in/stsoien/

Development Status :open_book:

At this stage, development has not yet commenced. However, significant groundwork has been completed to ensure a well-structured and feasible implementation of sBPF Program Execution on PolkaVM. 1. **Technical Design & Research** - We have formulated a detailed approach for executing sBPF programs in PolkaVM, which is documented in the article: **sBPF Program Execution on PVM - HackMD**. - This document outlines the key architectural decisions, execution flow, and potential challenges, particularly addressing the 4MB PolkaVM service limit. 2. Discussions with Key Experts - We have engaged in multiple discussions with Dr. Gavin Wood, both via email and in face-to-face meetings, to validate our approach. - These discussions have provided valuable insights into optimizing execution, handling state validation, and ensuring efficient integration with Polkadot's ecosystem. 3. Next Steps - With the technical research phase complete, we are now prepared to begin actual development. - The next milestone will focus on compiling and testing the feasibility of embedding sBPF in PolkaVM, followed by state validation and execution optimization.

Development Roadmap :nut_and_bolt:

Overview

This roadmap outlines the phased development of the sBPF Runtime as a Service in PolkaVM, ensuring that Solana smart contracts can execute efficiently within Polkadot's infrastructure. Given the 4MB size constraint of PVM services, the roadmap prioritizes feasibility analysis, optimization, and transaction processing integration. - **Total Estimated Duration:** 8 weeks - **Full-Time Equivalent (FTE):** 3 - **Total Costs:** $65,000 USD - **DOT %:** Percentage of Total Costs to be paid in vested DOT: 50%

Milestone: sBPF Runtime Feasibility & Prototype Development in PolkaVM

- **Estimated duration:** 8 weeks
- **FTE:** 3
- **Costs:** $65,000 USD

Number	Deliverable	Specification
0a.	License	Apache 2.0
0b.	Documentation	Initial architecture documentation explaining sBPF execution in PVM
0c.	Testing and Testing Guide	Unit tests to measure compiled sBPF interpreter size
0d.	Docker	Containerize and run Minimal sBPF Execution Test in PVM using Docker
0e.	Article	Publish an article outlining feasibility analysis and prototype development
1.	Compile `anza-xyz/sbpf` with 'no_std' and analyze feasibility	Analyze feasibility of `no_std` adaptation and remove Solana-specific dependencies
2.	Memory & Execution Profiling for Minimal sBPF	Measure compiled binary size, execution time, and memory footprint for minimal sBPF programs.
3.	Fork `anza-xyz/sbpf` & Introduce PVM Service (`jam-pvm-common`)	Refactor sBPF interpreter to work as a PolkaVM service using `jam-pvm-common`.
4.	Compile & Measure sBPF Binary Size	Ensure it fits within 4MB PVM service limit.
5.	Solana Account Model Integration	Port Solana's account model.
6.	Minimal sBPF Execution Test in PVM	Deploy and execute a simple contract (e.g., counter) in PolkaVM to confirm functional sBPF execution. The result of execution to be stored in a separated DB.

Future Plans

Lollipop is committed to expanding Polkadot's execution capabilities by enabling Solana sBPF program execution within PolkaVM. Our work on sBPF Runtime as a Service in PolkaVM is just the first step toward a broader cross-ecosystem compatibility strategy.

In the near future, we aim to:

1. Integration of Solana Account Model, Parallel Tx Scheduling, Interoperability & Compatibility - Parallel TX Scheduling: We think everything will work sequentially at this stage, yet we also believe we can port everything Solanas has into our PolkaVM-based runtime, like Parallel TX Scheduling, and we expect some obstacles and stumbling blocks for this issue, but will be properly addressed. - Interoperability & Compatibility: Our goal is to offer a commercial-ready system that's fully familiar to Solana developers. Apart from this grant, We're also integrating common Solana runtime dependencies (e.g., SPL libraries) to support standard interfaces with minimal changes. Our goal is to offer a commercial-ready system that's fully familiar to Solana developers. We are also planning Interoperability with Solana through lightweight instruction programs and post-execution callbacks, allowing NE-executed transactions to trigger on-chain logic.

2. Scale Up Development & Deployment - Complete the implementation and optimization of sBPF execution in PolkaVM with multiple PVM services, - Deploy this execution layer, ensuring efficient performance and seamless interaction with Solana's state model.

3. Enhance Compatibility & Performance - Investigate additional optimizations to improve the efficiency and security of sBPF

execution.

4. Expand Developer Adoption - Provide SDKs and tooling to help developers port and deploy Solana smart contracts to Polkadot. - Offer comprehensive documentation and examples to simplify integration.

5. Long-Term Vision: Unified Execution Layer - This initiative aligns with Lollipop's broader vision of creating a unified execution layer across multiple ecosystems. - In future iterations, we plan to extend PolkaVM's support beyond Solana's sBPF to other high-performance execution environments, such as AI-based computation or modular rollups.

Our goal is to position Lollipop as a key infrastructure provider, bridging execution environments across Polkadot, Solana, and beyond.

Additional Information :heavy_plus_sign:

- Lollipop is an innovative project focused on bridgeless scaling and execution infrastructure for Solana with Polkadot JAM. Unlike traditional Layer 2 solutions, Lollipop's Network Extensions (NEs) leverage cryptographic proofs and diff-based validation to ensure trustless execution without additional verification overhead.
- Our work on sBPF Runtime as a Service in PolkaVM represents a significant advancement in multi-chain execution compatibility, allowing Solana smart contracts to run seamlessly within Polkadot without needing a bridge.
- This initiative has already gained attention within the Polkadot and Solana communities,as we move forward, we look to collaborate with both the Polkadot and Solana ecosystems to ensure this technology benefits developers, validators, and users alike.